



INFORME

I. PORTADA

Tema:	Recorridos Inorden, Preorden, Postorden y BFS
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumno:	Medina Chico Darlyn Monserrath
Asignatura:	Estructura de datos
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

Objetivo General:

Implementar y analizar los principales recorridos de árboles binarios utilizando C++ y Java, aplicando estructuras de datos dinámicas, recursividad y colas.

Objetivos Específicos:

- Explicar la diferencia entre los recorridos DFS (Inorden, Preorden, Postorden) y BFS.
- Implementar los recorridos Inorden, Preorden y Postorden utilizando recursividad.
- Implementar el recorrido BFS (amplitud) utilizando una estructura de cola.
- Comparar los resultados y el comportamiento entre las implementaciones en C++ y Java.
- Aplicar los conceptos de recorridos a un caso real relacionado con el proyecto final de la materia.

2.2 Desarrollo Teórico

Introducción

Un recorrido de árbol es un proceso sistemático que permite visitar cada uno de los nodos que componen una estructura jerárquica exactamente una vez. [1] Los árboles binarios son estructuras de datos no lineales ampliamente utilizadas en informática para representar relaciones jerárquicas como sistemas de archivos, expresiones matemáticas, estructuras organizacionales y módulos de software.

La importancia de los recorridos radica en que permiten procesar la información almacenada en el árbol de diferentes maneras, dependiendo del orden en que se visitan los nodos. En esta práctica se estudian cuatro tipos fundamentales de recorridos: Inorden, Preorden, Postorden (técnicas de búsqueda en profundidad o DFS) y BFS (búsqueda en amplitud o nivel por nivel).

Marco Teórico

Un árbol binario es una estructura de datos jerárquica en la cual cada nodo puede tener como máximo dos hijos: uno izquierdo y uno derecho. Se utiliza ampliamente en informática para representar relaciones jerárquicas, búsquedas eficientes y organización de información.



Recorridos de Árboles Binarios

Los recorridos son métodos sistemáticos para visitar todos los nodos de un árbol [2] [5]:

1. Recorrido Inorden (Izquierda → Raíz → Derecha)

Se aplica en Árboles Binario de Búsqueda (BST), produce los elementos en orden ascendente. Es útil para ordenar datos y para obtener listados secuenciales.

2. Recorrido Preorden (Raíz → Izquierda → Derecha)

Es útil para mostrar la estructura jerárquica de un sistema, como menús o módulos, ya que primero se visita el nodo principal y luego sus submódulos.

3. Recorrido Postorden (izquierda → Derecha → Raíz)

Se aplica cuando es necesario procesar primero los nodos hijos antes que el padre, por ejemplo, al eliminar un árbol o calcular dependencias.

4. Recorrido BFS (Breadth-First Search):

Se visitan todos los nodos nivel por nivel, de izquierda a derecha. Utiliza una cola (estructura FIFO) para almacenar los nodos pendientes de procesar. Es ideal para mostrar información en capas o niveles jerárquicos.

Comparación DFS vs BFS [3] [4]:

Característica	DFS (Profundidad)	BFS (Amplitud)
Estructura auxiliar	Pila (implícita por recursividad)	Cola (explícita)
Orden de visita	Va hasta el fondo antes de retroceder	Va nivel por nivel
Uso de memoria	O(altura del árbol)	O(ancho del árbol)

Metodología

La práctica se desarrolló siguiendo un enfoque teórico-práctico, combinando el estudio de los recorridos de árboles binarios con su implementación en lenguajes de programación. El proceso metodológico se organizó en las siguientes fases:

1. Planificación – Se analizó la guía del docente y se definió la estructura del repositorio GitHub, organizando el código fuente en src/cpp/ y src/java/, las capturas en capturas/, el informe en informes/ y los ejercicios resueltos dentro del propio código comentado.

2. Entorno de trabajo – Se configuraron los compiladores (MinGW para C++ y JDK para Java) y se verificó la ejecución del código base. Se solucionaron inconvenientes con la codificación de



archivos en VS Code (error de la "U" UTF-8 con BOM) y con la configuración del PATH de MinGW.

3. Implementación – Se programaron los recorridos Inorden, Preorden, Postorden (recursivos) y BFS (con cola) en un solo archivo por lenguaje. Se resolvieron los ejercicios integrados en el mismo código: recorridos del árbol inicial (Ejercicio 1), inserción de 5 nodos nuevos (1, 3, 18, 25, 30) para el Ejercicio 2, funciones para contar nodos (Ejercicio 3) y contar hojas (Ejercicio 4), y representación del sistema web como árbol binario (Ejercicio 5).

4. Caso real – Se modelaron los módulos del sistema web, seleccionando Preorden para mostrar el menú principal, Postorden para procesar módulos internos antes que el principal, y BFS para la navegación nivel por nivel.

5. Evidencias – Se capturaron las ejecuciones en C++ y Java, documentando todo en el informe académico, en el README.md del repositorio y organizando las capturas en la carpeta capturas/.

6. Comparación – Se contrastaron las implementaciones en ambos lenguajes, analizando diferencias en sintaxis, manejo de punteros (C++) vs referencias (Java), gestión de memoria manual vs automática, y uso de estructuras dinámicas como colas.

2.3 Desarrollo y Ejercicios Resueltos

Ejercicio 1: Recorridos del árbol inicial.

```
      10
     /  \
    5    15
   /\  /\
  2 7 12 20
```

Preorden : 10, 5, 2, 7, 15, 12, 20

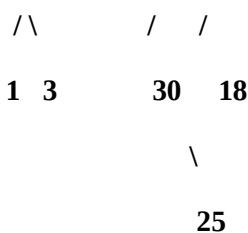
Inorden : 2, 5, 7, 10, 12, 15, 20

Postorden: 2, 7, 5, 12, 20, 15, 10

BFS (Nivel por nivel): 10, 5, 15, 2, 7, 12, 20

Ejercicio 2: Árbol modificado con nodos 1, 3, 18 y 25, 30

```
      10
     /   \
    5     15
   /\    /\
  2  7  12 20
```



Preorden: 10, 5, 2, 1, 3, 7, 15, 12, 30, 20, 18, 25

Inorden: 1, 2, 3, 5, 7, 10, 12, 15, 18, 20, 25, 30

Postorden: 1, 3, 2, 7, 5, 30, 12, 25, 18, 20, 15, 10

BFS: 10, 5, 15, 2, 7, 12, 20, 1, 3, 30, 18, 25

Ejercicio 3: Función para contar nodos.

Explicación del algoritmo: La función utiliza recursividad. Por cada nodo, suma 1 (el nodo actual) más el resultado de contar los nodos del subárbol izquierdo y del subárbol derecho. Cuando llega a un nodo nulo, retorna 0.

Código en c++:

```
int contarNodos(Nodo* raiz) {  
    if (raiz == nullptr) return 0;  
    return 1 + contarNodos(raiz->izquierda) + contarNodos(raiz->derecha);  
}
```

// Ejemplo de uso

```
cout << "Total de nodos: " << contarNodos(raiz) << endl;
```

Código en java:

```
public static int contarNodos(Nodo raiz) {  
    if (raiz == null) return 0;  
    return 1 + contarNodos(raiz.izquierda) + contarNodos(raiz.derecha);  
}
```

// Ejemplo de uso

```
System.out.println("Total de nodos: " + contarNodos(raiz));
```

Ejercicio 4: Función para contar hojas.



Explicación del algoritmo: La función recorre el árbol recursivamente. Si un nodo es nulo, retorna 0. Si un nodo no tiene hijo NI derecho, es una hoja y retorna 1. En caso contrario, suma el resultado de contar hojas en ambos subárboles.

Código en c++:

```
int contarHojas(Nodo* raiz) {  
    if (raiz == nullptr) return 0;  
  
    if (raiz->izquierda == nullptr && raiz->derecha == nullptr) return 1;  
  
    return contarHojas(raiz->izquierda) + contarHojas(raiz->derecha);  
}
```

// Ejemplo de uso

```
cout << "Total de hojas: " << contarHojas(raiz) << endl;
```

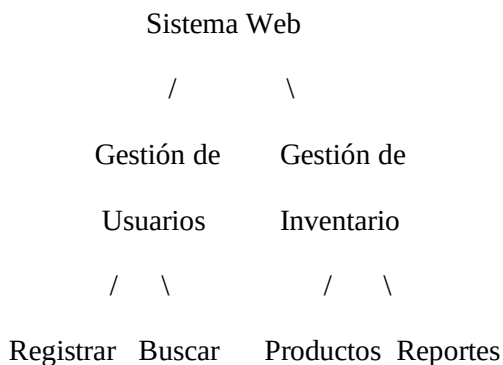
Código en java:

```
public static int contarHojas(Nodo raiz) {  
    if (raiz == null) return 0;  
  
    if (raiz.izquierda == null && raiz.derecha == null) return 1;  
  
    return contarHojas(raiz.izquierda) + contarHojas(raiz.derecha);  
}
```

// Ejemplo de uso

```
System.out.println("Total de hojas: " + contarHojas(raiz));
```

Ejercicio 5: Representación del sistema web como árbol binario.



- **Menú principal:** Se usa Preorden, ya que primero se muestra el nodo raíz (Sistema Web) y luego sus submódulos.



- **Procesar módulos internos:** Se usa Postorden, porque primero se procesan los hijos antes que el padre.
- **Mostrar nivel por nivel:** Se usa BFS, ya que permite recorrer los módulos por capas jerárquicas.

2.4 Resultados y Capturas

Ejercicio 1: Recorridos manuales del árbol base

Ejecución:

Java

```
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_recorridos_arboles_uta_89e9c8b1\bin> cd src\cpp
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_recorridos_arboles_uta_89e9c8b1\bin> java -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_recorridos_arboles_uta_89e9c8b1\bin' Main
=====
EJERCICIO 1 - ÁRBOL INICIAL
=====
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
```

C++

```
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_recorridos_arboles_uta> cd src\cpp
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_recorridos_arboles_uta\src\cpp> g++ main.cpp
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_recorridos_arboles_uta\src\cpp> .\main.exe
=====
EJERCICIO 1 - ÁRBOL INICIAL
=====
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
```

Ejercicio 2: Árbol ampliado (nodos 1, 3, 18, 25, 30)

Ejecución:

Java

```
=====
EJERCICIO 2 - ÁRBOL CON NODOS AGREGADOS
Nodos agregados: 1, 3, 18, 25 + nodo extra 30
=====
Preorden: 10 5 2 1 3 7 15 12 30 20 18 25
Inorden: 1 2 3 5 7 10 12 30 15 18 25 20
Postorden: 1 3 2 7 5 30 12 25 18 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 30 18 25
```

C++



```
=====
EJERCICIO 2 - ARBOL CON NODOS AGREGADOS
Nodos agregados: 1, 3, 18, 25, 30
=====
Preorden: 10 5 2 1 3 7 15 12 30 20 18 25
Inorden: 1 2 3 5 7 10 12 30 15 18 25 20
Postorden: 1 3 2 7 5 30 12 25 18 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 30 18 25
```

Ejercicio 3: Función para contar nodos.

Ejecución:

Java

```
=====
EJERCICIO 3 - CONTAR NODOS
=====
Total de nodos en el árbol: 12
```

C++

```
=====
EJERCICIO 3 - CONTAR NODOS
=====
Total de nodos en el arbol: 12
```

Ejercicio 4: Función para contar hojas.

Ejecución:

Java

```
=====
EJERCICIO 4 - CONTAR HOJAS
=====
Total de hojas en el árbol: 5
(Hojas: nodos sin hijos: 1, 3, 7, 12, 25, 30)
```

C++



EJERCICIO 4 - CONTAR HOJAS

Total de hojas en el árbol: 5
(Hojas: nodos sin hijos: 1, 3, 7, 12, 25, 30)

Ejercicio 5: Representación del sistema web como árbol binario.

Ejecución:

Java

```
jdk-21.0.11.10-hotspot\bin\java.exe -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\Usuario\AppData\Roaming\Code\9ca497e100971757c6\redhat.java\jdk_ws\tarea2.2_repo_recorridos_arboles_uta_09e9c8b1\bin" Main
```

EJERCICIO 5 - SISTEMA WEB
Aplicación al proyecto final

Estructura del Sistema Web:

```
graph TD
    SW[Sistema Web] --> GU[Gestión de Usuarios]
    SW --> GI[Gestión de Inventario]
    GU --> R[Registrar]
    GU --> B[Buscar]
    GI --> P[Productos]
    GI --> Re[Reportes]
```

--- RESULTADOS DE RECORRIDOS EN EL SISTEMA WEB ---

NOTA: Los resultados mostrados corresponden a la estructura numerica aplicando la misma logica a los modulos del sistema web:

- Preorden: Sistema Web -> Gestion Usuarios -> Registrar -> Buscar -> Gestion Inventario -> Productos -> Reportes
- Postorden: Registrar -> Buscar -> Gestion Usuarios -> Productos -> Reportes -> Gestion Inventario -> Sistema Web
- BFS: Sistema Web -> Gestion Usuarios -> Gestion Inventario -> Registrar -> Buscar -> Productos -> Reportes

C++

```
EJERCICIO 5 - SISTEMA WEB
Aplicación al proyecto final
```

Estructura del Sistema Web:

```
graph TD
    SW[Sistema Web] --> GU[Gestión de Usuarios]
    SW --> GI[Gestión de Inventario]
    GU --> R[Registrar]
    GU --> B[Buscar]
    GI --> P[Productos]
    GI --> Re[Reportes]
```

--- RESULTADOS DE RECORRIDOS EN EL SISTEMA WEB ---

NOTA: Los resultados mostrados corresponden a la estructura numerica aplicando la misma logica a los modulos del sistema web:

- Preorden: Sistema Web -> Gestion Usuarios -> Registrar -> Buscar -> Gestion Inventario -> Productos -> Reportes
- Postorden: Registrar -> Buscar -> Gestion Usuarios -> Productos -> Reportes -> Gestion Inventario -> Sistema Web
- BFS: Sistema Web -> Gestion Usuarios -> Gestion Inventario -> Registrar -> Buscar -> Productos -> Reportes

2.5 Preguntas de Reflexión

1. ¿Qué recorrido sirve para ordenar valores en un BST?

El recorrido Inorden es el que sirve para ordenar valores en un Árbol Binario de Búsqueda (BST), ya que visita primero el subárbol izquierdo (valores menores), luego la raíz y finalmente el



subárbol derecho (valores mayores), produciendo una secuencia ascendente de todos los elementos almacenados en el árbol.

2. ¿Qué diferencia existe entre DFS y BFS?

DFS (Preorden, Inorden, Postorden) explora en profundidad usando recursividad, mientras que BFS explora nivel por nivel usando una cola. En el sistema web, DFS iría hasta "Registrar" antes de ver "Inventario", mientras que BFS mostraría primero "Sistema Web", luego "Usuarios" e "Inventario", y después "Registrar", "Buscar", "Productos" y "Reportes".

3. ¿Por qué BFS requiere una cola?

BFS requiere una cola porque necesita procesar los nodos en orden FIFO, asegurando que los nodos del nivel actual se recorran completamente antes de pasar al siguiente nivel, lo que permite mostrar el sistema web nivel por nivel: Nivel 0 → Sistema Web, Nivel 1 → Usuarios e Inventario, Nivel 2 → Registrar, Buscar, Productos y Reportes.

4. ¿En qué caso real se puede usar Preorden?

Preorden es ideal para mostrar un menú principal con estructura jerárquica completa, como el menú de navegación del sistema web: Sistema Web → Usuarios → Registrar → Buscar → Inventario → Productos → Reportes. Esto permite al usuario ver primero el módulo principal y luego los submódulos en orden de aparición.

5. ¿En qué caso real se puede usar Postorden?

Postorden es útil para procesar módulos internos antes que el módulo principal, como al cerrar sesión: primero cerrar submódulos (Registrar, Buscar, Productos, Reportes), luego módulos principales (Usuarios, Inventario) y finalmente el Sistema Web. Esto garantiza que todos los procesos internos finalicen correctamente antes de salir del sistema principal.

2.6 Preguntas tipo Moodle

1. ¿Cuál es el orden del recorrido Inorden?

- A. Raíz, izquierda, derecha
- B. Izquierda, raíz, derecha
- C. Izquierda, derecha, raíz
- D. Nivel por nivel

Respuesta: B

2. ¿Qué estructura utiliza BFS?

- A. Pila
- B. Lista circular
- C. Cola
- D. Árbol AVL

Respuesta: C

3. ¿Cuál recorrido visita primero la raíz?



- A. Inorden
- B. Preorden
- C. Postorden
- D. BFS únicamente

Respuesta: B

4. ¿Cuál recorrido procesa la raíz al final?

- A. Preorden
- B. Inorden
- C. Postorden
- D. Nivel por nivel

Respuesta: C

5. En un BST, el recorrido Inorden permite obtener:

- A. Elementos desordenados
- B. Elementos por niveles
- C. Elementos en orden ascendente
- D. Solo hojas

Respuesta: C

Pregunta práctica:

Complete el código C++ del recorrido inorden:

```
void inorden(Nodo* raiz) {  
  
    if (raiz == nullptr) return;  
  
    inorden(raiz->izquierda); // Subárbol izquierdo  
  
    cout << raiz->dato << " ";  
  
    inorden(raiz->derecha); // Subárbol derecho  
  
}
```

Respuesta: inorden

2.7 Comparación C++ vs Java

En esta sección se analizan las principales diferencias entre las implementaciones realizadas en C++ y Java para los recorridos de árboles binarios, considerando aspectos como sintaxis, manejo de memoria, estructuras de datos y rendimiento.

Aspecto	C++	Java
Declaración de nodos	Uso de <code>struct</code> o <code>class</code> con punteros (*)	Uso de <code>class</code> con referencias implícitas



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Punteros	Explícitos (*, ->, &, nullptr)	Inexistentes (se usan referencias automáticas)
Manejo de memoria	Manual (new / delete)	Automático (Garbage Collector)
Recursividad	Soporte nativo, riesgo de desbordamiento de pila	Soporte nativo, optimizado por JVM
Cola para BFS	#include <queue> usando queue<Nodo*>	import java.util.Queue usando LinkedList<>()
Sintaxis de impresión	cout << valor << " ";	System.out.print(valor + " ");
Compilación	g++ main.cpp -o programa	javac Main.java
Ejecución	./programa (Linux) o programa.exe (Windows)	java Main
Velocidad de ejecución	Más rápido (código nativo)	Más lento (bytecode + JIT)
Portabilidad	Requiere recompilación por plataforma	Bytecode multiplataforma (JVM)
Depuración	Herramientas como GDB	Depurador integrado en IDE (Eclipse, VS Code, IntelliJ)

Comparación de código: Creación de un nodo

C++

```
struct Nodo {  
  
    int dato;  
  
    Nodo* izquierda;  
  
    Nodo* derecha;  
  
    Nodo(int valor) {  
  
        dato = valor;  
  
        izquierda = nullptr;  
  
        derecha = nullptr;  
  
    }  
  
};  
  
// Crear nodo en el heap
```



```
Nodo* nodo = new Nodo(10);
```

```
// Liberar memoria
```

```
delete nodo;
```

Java

```
class Nodo {
```

```
    int dato;
```

```
    Nodo izquierda;
```

```
    Nodo derecha;
```

```
    Nodo(int valor) {
```

```
        this.dato = valor;
```

```
        this.izquierda = null;
```

```
        this.derecha = null;
```

```
    }
```

```
}
```

```
// Crear nodo (no hay punteros)
```

```
Nodo nodo = new Nodo(10);
```

```
// La memoria se libera automáticamente (GC)
```

Comparación de código: Recorrido BFS (cola)

C++

```
#include <queue>
```

```
using namespace std;
```

```
void bfs(Nodo* raiz) {
```

```
    queue<Nodo*> cola;
```

```
    cola.push(raiz);
```

```
    while (!cola.empty()) {
```



```
Nodo* actual = cola.front();

cola.pop();

cout << actual->dato << " ";

if (actual->izquierda != nullptr)

    cola.push(actual->izquierda);

if (actual->derecha != nullptr)

    cola.push(actual->derecha);

}

}
```

Java

```
import java.util.LinkedList;

import java.util.Queue;

public static void bfs(Nodo raiz) {

    Queue<Nodo> cola = new LinkedList<>();

    cola.add(raiz);

    while (!cola.isEmpty()) {

        Nodo actual = cola.poll();

        System.out.print(actual.dato + " ");

        if (actual.izquierda != null)

            cola.add(actual.izquierda);

        if (actual.derecha != null)

            cola.add(actual.derecha);

    }

}
```



Lenguaje	Ventajas	Desventajas
C++	• Mayor velocidad de ejecución • Control total sobre la memoria • Ideal para sistemas embebidos o de alto rendimiento • Menor consumo de recursos	• Gestión manual de memoria (propensa a errores) • Sintaxis más compleja (punteros) • Menos portabilidad • Mayor riesgo de memory leaks
Java	• Gestión automática de memoria (GC) • Sintaxis más limpia y simple • Alta portabilidad (JVM) • Menor riesgo de errores de memoria • Gran cantidad de librerías estándar	• Menor rendimiento que C++ • Mayor consumo de RAM • Mayor tiempo de inicio (JVM) • Latencia por pausas del GC

2.8 Conclusiones

- Se logró implementar correctamente los cuatro tipos de recorridos (Inorden, Preorden, Postorden y BFS) en ambos lenguajes, comprendiendo sus diferencias y aplicaciones prácticas.
- El recorrido Inorden resultó útil para obtener datos ordenados en un BST, Preorden para mostrar menús jerárquicos, Postorden para procesar módulos internos antes que el principal y BFS para visualizar estructuras por niveles.
- Se evidenció la diferencia fundamental entre DFS (profundidad, usando recursividad) y BFS (amplitud, usando cola), así como la importancia de elegir el recorrido adecuado según el caso real.
- La implementación en Java resultó más sencilla por su gestión automática de memoria, mientras que C++ ofreció mayor control y rendimiento, pero con mayor complejidad en el manejo de punteros.
- El uso de GitHub permitió organizar el código, documentar el proceso y trabajar de manera colaborativa, facilitando la entrega de evidencias y el seguimiento del proyecto.

2.9 Recomendaciones

- Siempre documentar el código con comentarios claros que expliquen la lógica de cada función, especialmente en los recorridos recursivos y en el manejo de la cola para BFS, facilitando así la comprensión y el mantenimiento del programa.
- Dibujar el árbol binario en papel o en una herramienta digital antes de implementarlo, ya que visualizar la estructura ayuda a anticipar los resultados de cada recorrido y permite detectar errores de forma más rápida durante la depuración.
- Probar cada recorrido por separado antes de integrarlos en el programa final, ejecutando individualmente Inorden, Preorden, Postorden y BFS para verificar que cada uno funcione correctamente con diferentes casos de prueba.
- Gestionar correctamente la memoria en C++ utilizando `delete` para liberar los nodos creados con `new`, evitando así pérdidas de memoria (memory leaks) que puedan afectar el rendimiento del programa.
- Utilizar un sistema de control de versiones como Git, manteniendo commits frecuentes con mensajes descriptivos y subiendo todas las evidencias (código fuente, capturas de pantalla, README actualizado) al repositorio grupal en GitHub.



2.10 Repositorio GitHub

La estructura final del repositorio quedó organizada de la siguiente manera:

TAREA2.2_REPO_RECORRIDOS_ARBOLES_UTA/

```
|
|— docs/
|— exercises/
|— moodle/
|— src/
|   |— cpp/
|   |   |— main.cpp          # Código completo en C++ (todos los ejercicios)
|   |   |— java/
|   |       |— Main. java      # Código completo en Java (todos los ejercicios)
|— capturas/
|   |— cpp_ejecucion_Ejer_1_2_3_4.png    # Captura de ejecución en C++
|   |— cpp_ejecucion_Ejer_5.png
|   |— java_ejecucion_Ejer_1_2_3_4.png    # Captura de ejecución en Java
|   |— java_ejecucion_Ejer_5.png
|— INFORME_PRACTICA.pdf
|— .gitignore
|— LICENSE
|— README.md                # Actualizado
```

2.11 Referencias bibliográficas

- [1] K. C. Cedeño Zambrano, D. D. Lucas Alay, L. A. Verá Polanco, J. M. Vera Zambrano, and V. F. Romero Castro, "Importancia de árboles binarios en la programación," *Journal TechInnovation*, vol. 1, no. 2, pp. 33–42, Jul. 2022.
- [2] "Recorridos (preorden, inorden y postorden)," *Tutoriales Programación Ya*. [En línea]. Disponible: <https://www.programacionya.com>. [Accedido: 07-may-2026].
- [3] "Exploración de Árboles en Java: BFS, DFS con ejemplos," *CiberAula*. [En línea]. Disponible: <https://www.ciberaula.com>. [Accedido: 07-may-2026].
- [4] "¿Cuáles son los algoritmos más comunes para atravesar un árbol?," *LinkedIn Learning*. [En línea]. Disponible: <https://es.linkedin.com/learning>. [Accedido: 07-may-2026].
- [5] "Aplicaciones de los árboles binarios," in *Apuntes de Programación*, IES San Clemente. [En línea]. Disponible: <https://www.iessanclemente.net>. [Accedido: 07-may-2026].